

Timing Analysis of Counter using OpenSTA

INTRODUCTION:

In this lab, students will learn how to perform static timing analysis on a synthesized counter using OpenSTA. They will practice importing the gate-level netlist from Yosys, loading timing constraints, linking standard cell libraries, and generating detailed timing reports. This exercise demonstrates how timing is verified for hardware-efficient logic at the signoff level.

APPLICATION:

Synchronous 4-bit counters are used in frequency division, digital clocks, timers, and event counting. They also serve as building blocks for registers, sequence generators, and other complex digital circuits.

OBJECTIVE:

After completing this lab, you will be able to:

- Import a gate-level netlist and standard cell library into OpenSTA (OSU018 lib).
- Load and apply timing constraints (e.g., clock, input/output delays) using SDC format.
- Perform static timing analysis to check setup and hold margins across the design.
- Understand timing reports, including slack values, critical paths, and violation detection.
- Interpret timing analysis results to guide optimization of digital designs for timing closure.

PROCEDURE TO CREATE A LAB:

Step 1: Set up Project Directory

1.1: Create a directory for your lab: `mkdir <project_name>`

1.2: Open lab directory: `cd <project_name>`

1.3: Create directory for project: `mkdir counter`

1.4: Open project directory: `cd counter`

Step 2: Create Design file

2.1: Open gvim editor for design with command: `gvim counter.v`

RTL Design Code:

```
module counter ( input clk,
                 input rst,
                 output reg[3:0] out );
always @ (posedge clk) begin
if (! rst)
out <= 0;
else
out <= out + 1;
end
endmodule
```

2.2: save the file using command- :wq

Step 3: Create .sdc file

3.1: Open gvim editor for the SDC with command: gvim counter.sdc

For 10ns:

```
create_clock -period 10 -name clk [get_ports clk]
set_input_delay 0 -min -rise [get_ports clk] -clock clk
set_input_delay 5 -max -fall [get_ports rst] -clock clk
set_input_transition 10 -min -rise [get_ports rst] -clock clk
set_input_transition 25 -max -fall [get_ports clk] -clock clk
set_load -pin_load 4 [get_ports out]
set_output_delay 2 -min -rise [get_ports out] -clock clk
```

Note: For 5ns, change the clock period to 5ns in sdc.

Step 4: Static Timing Analysis for the design with OpenSTA

Invoke OpenSTA in the terminal using the command: sta

4:1 Use the following commands:

Loads the liberty to osu018 standard cell library

i) read_liberty osu018_stdcells.lib

Reads your synthesized gate-level Verilog netlist

ii) read_verilog counter_synth.v

Links the loaded netlist and library to the top module name

iii) link_design counter

Loads your timing constraints from an SDC file

iv) read_sdc counter.sdc

Runs timing analysis to check all timing constraints and generates a report.

v) report_checks

OBSERVATION:

- Detect setup and hold timing violations
- Examine input and output delay constraints
- Review total negative slack and worst slack reports

OUTPUT:

Reports:

For 10ns:

→ for setup

```
OpenSTA> report_checks
Startpoint: rstn (input port clocked by clk)
Endpoint: _21_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max
```

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
5.00	5.00 v	input external delay
0.00	5.00 v	rstn (in)
4.07	9.07 ^	_14_/Y (OAI21X1)
0.18	9.26 v	_15_/Y (AOI21X1)
0.00	9.26 v	_21_/D (DFFPOSX1)
	9.26	data arrival time
10.00	10.00	clock clk (rise edge)
0.00	10.00	clock network delay (ideal)
0.00	10.00	clock reconvergence pessimism
-0.29	10.00 ^	_21_/CLK (DFFPOSX1)
	9.71	library setup time
	9.71	data required time
	9.71	data required time
	-9.26	data arrival time
	0.46	slack (MET)

→ for hold

```
Startpoint: rstn (input port clocked by clk)
Endpoint: _21_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: min
```

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00 ^	input external delay
0.00	0.00 ^	rstn (in)
-0.18	-0.18 v	_14_/Y (OAI21X1)
0.06	-0.13 ^	_15_/Y (AOI21X1)
0.00	-0.13 ^	_21_/D (DFFPOSX1)
	-0.13	data arrival time
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	clock reconvergence pessimism
0.00	0.00 ^	_21_/CLK (DFFPOSX1)
	0.00	library hold time
	0.00	data required time
	0.00	data required time
	0.13	data arrival time
	-0.13	slack (VIOLATED)

For 5ns:

→ [for setup](#)

```

OpenSTA> report_checks
Startpoint: rstn (input port clocked by clk)
Endpoint: _21_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max

```

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
5.00	5.00	input external delay
0.00	5.00	v rstn (in)
4.07	9.07	^ _14_/Y (OAI21X1)
0.18	9.26	v _15_/Y (AOI21X1)
0.00	9.26	v _21_/D (DFFPOSX1)
	9.26	data arrival time
5.00	5.00	clock clk (rise edge)
0.00	5.00	clock network delay (ideal)
0.00	5.00	clock reconvergence pessimism
	5.00	^ _21_/CLK (DFFPOSX1)
-0.29	4.71	library setup time
	4.71	data required time
	4.71	data required time
	-9.26	data arrival time
	-4.54	slack (VIOLATED)

→ [for hold](#)

```

Startpoint: rstn (input port clocked by clk)
Endpoint: _21_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: min

```

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	^ input external delay
0.00	0.00	^ rstn (in)
-0.18	-0.18	v _14_/Y (OAI21X1)
0.06	-0.13	^ _15_/Y (AOI21X1)
0.00	-0.13	^ _21_/D (DFFPOSX1)
	-0.13	data arrival time
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	clock reconvergence pessimism
	0.00	^ _21_/CLK (DFFPOSX1)
0.00	0.00	library hold time
	0.00	data required time
	0.00	data required time
	0.13	data arrival time
	-0.13	slack (VIOLATED)

COMPARISION:

Timing	Setup	Hold
10ns	0.46	-0.13
5ns	-4.54	-0.13

Note: Timing can be met by changing constraints in sdc.

CONCLUSION:

OpenSTA verifies the timing of the synthesized 4-bit counter design using gate-level netlist and timing constraints. The analysis confirms that all critical paths meet setup and hold requirements, with positive slack values. No timing violations are detected, ensuring reliable operation at the specified clock frequency. The clock, reset, and counter increment logic show correct timing relationships in the reports. Overall, OpenSTA validates that the hardware counter will function accurately in real digital systems.